This project is from CS 360 Mobile Architecture and Design and is an inventory app I decided to enhance for my capstone. This enhancement targets algorithms and data structures used in this project.

I still wish to pursue a career in app development, so this project is a good fit. I selected this item because I felt it was a good fit for my goals and career aspirations that can be improved easily. It highlights my system design skill; how I can create new files that can scale and add to the project. It also expresses my skills in the creation and use of multiple algorithms and how they are applied to the project.

The artifact was improved via the introduction of a sorting and filtering algorithm. The data structure was also changed to a List<InventoryList> when working with the actual data. To achieve these, InventoryAlgorithms.kt was created that has a low stock filter at O(n) and a merge sorting algorithm as O(n log n). I also added unit testing for the algorithms. InventoryViewModel.kt was also created, which primarily handles most of the inventory displays. I then began to implement the ViewModel into ActivityMain.kt. I also quickly wrote up a UML class lucidchart of the entire project.

Now, what I did not do yet. There was no substantial enhancements toward UI/UX. Buttons were added, but they were last minute at best to test my algorithms. The existing display remains mainly unchanged. I also did not fully migrate CRUD operations from ActivityMain to InventoryViewModel. CRUD functionality still sits in DatabaseHelper and InventoryViewModel acts as a display for the current database. Finally, I did not create a user-changeable threshold for the low stock filter. This was mainly due to testing and its first iteration.

While I was working on this enhancement for the artifact, I realized the importance of designing a program before coding. I created the UML class diagram before any coding, and I made that a restriction of mine so it would get done. I felt so relaxed while coding because I knew what I needed to do and how to start. It reminded me when I first started this class. I had to create some diagrams basically showing how I wanted the screens to look, some UML stuff, etcetera. I remember how much I enjoy that. I also caught myself appreciating how intuitive the Kotlin language has been so far. Both syntax and logic makes a lot of sense to me, it just clicks.